

Assignment 02 – Secure Systems

S4129857

Part One

For this assignment, I have selected two of the most security-critical subsystems of the Electronic Voting Platform:

1. Online voting client
 - Used by voters at home or in polling stations to authenticate, view candidates and cast votes.
2. Vote storage and tallying service
 - Central backend system that securely stores all submitted votes and calculates results.

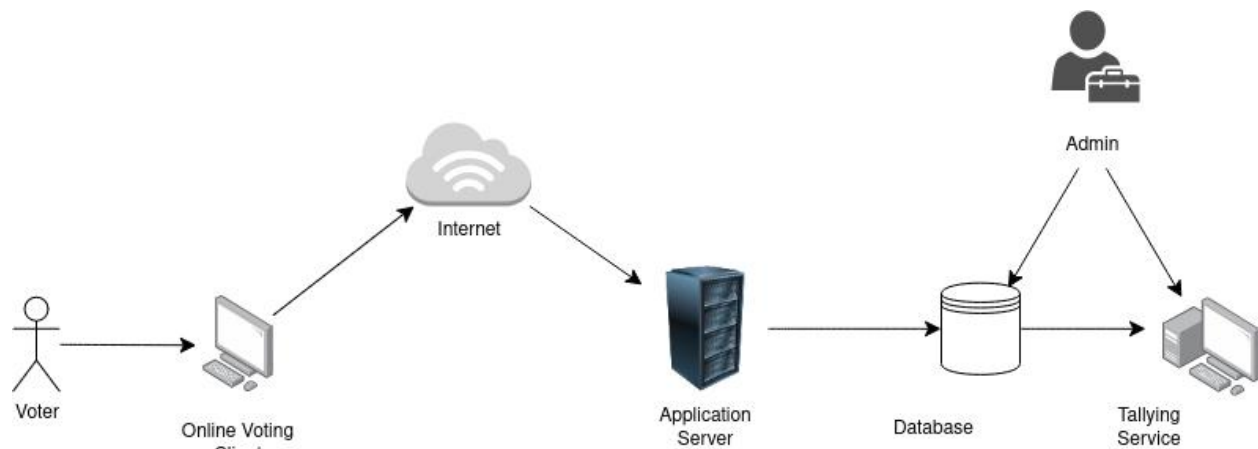


Figure 1.1: System Context

Threat modelling method

I chose a threat modelling method combining STRIDE for threat identification and DREAD for risk prioritization.

- **STRIDE**
 1. STRIDE is a Microsoft threat model widely used to categorize security threats:
 - Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service and Elevation of Privilege.
- **DREAD**
 1. DREAD scoring can be used to prioritize threats:

- Damage, Reproducibility, Exploitability, Affected users and Discoverability.
- Each of the 5 dimensions is scored from 1 to 10, making the total risk score between 1 to 50.
 1. Damage Potential: How Much Damage Could the Attack Cause?
 - 0: No damage
 - 5: Information disclosure
 - 8: Non-sensitive user data related to individuals or employer compromised
 - 9: Non-sensitive administrative data compromised
 - 10: Destruction of an information system; data or application unavailability
 2. Reproducibility: How Easily Can the Attack Be Reproduced?
 - 0: Difficult or impossible
 - 5: Complex
 - 7.5: Easy
 - 10: Very easy
 3. Exploitability: What's Required to Launch the Attack?
 - 2.5: Advanced programming and networking skills
 - 5: Available attack tools
 - 9: Web application proxies
 - 10: Web browser
 4. Affected Users: How Many People Would the Attack Affect?
 - 0: No users
 - 2.5: Individual user
 - 6: Few users
 - 8: Administrative users
 - 10: All users
 5. Discoverability: How Easy Is the Vulnerability to Discover?
 - 0: Hard to discover the vulnerability
 - 5: HTTP requests can uncover the vulnerability
 - 8: Vulnerability found in the public domain
 - 10: Vulnerability found in web address bar or form
- This allows ranking threats as High, Medium and Low.
 1. Critical (40–50): Critical vulnerability; address immediately.
 2. High (25–39): Severe vulnerability; consider for review and resolution soon.

3. Medium (11–24): Moderate risk; review after addressing severe and critical risks.
4. Low (1–10): Low risk to infrastructure and data.

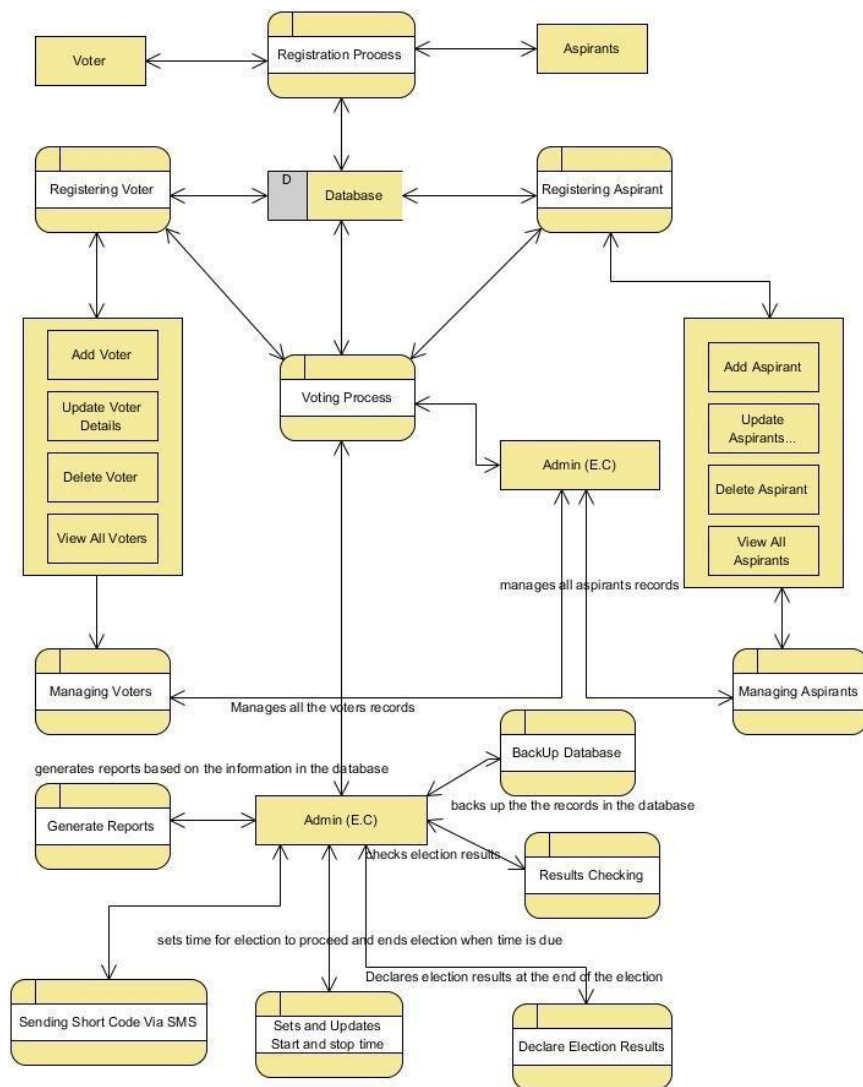
- **Tailored steps applied:**

1. Define assets & scope:

- Identify what needs protection: voting client, vote storage database and vote tallying process.
- Define security objectives: confidentiality, integrity, availability and authenticity of votes.

2. Draw architectural diagrams:

- Create Data Flow Diagrams (DFDs) showing major components: client, web server, application logic, database and tallying process.



Reference: Researchgate

Figure 1.2: Data Flow Diagram

3. Identify trust boundaries:

- Identify places where data crosses from one trust level to another.
- These are the key points where STRIDE threats are most relevant.

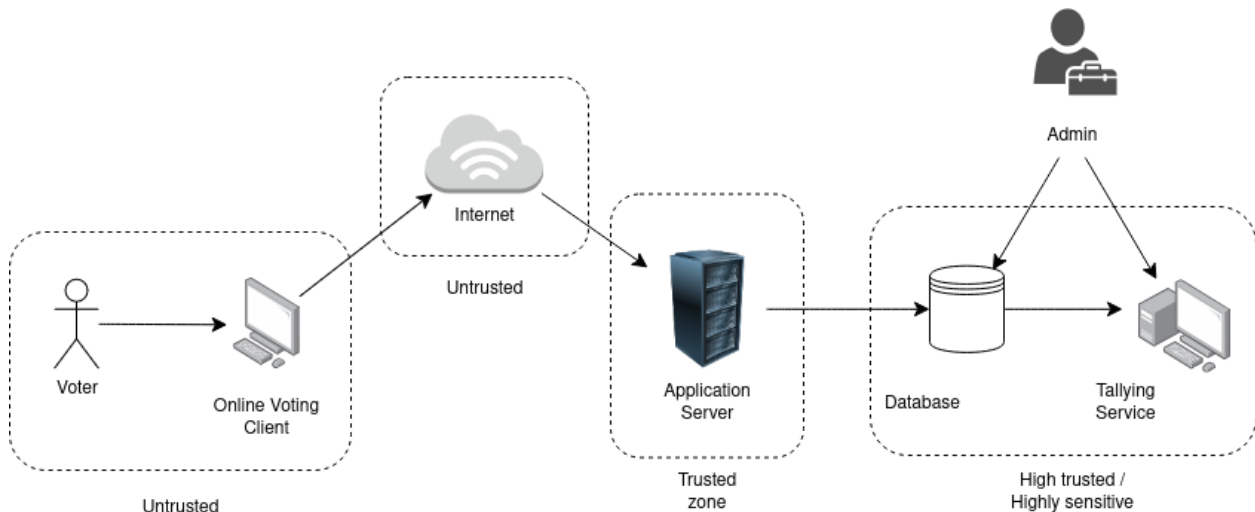


Figure 1.3: Trust Boundaries

4. Apply STRIDE to each DFD element:

- For each process, datastore and data flow, apply STRIDE categories to systematically identify potential threats.

5. Enumerate threats:

- List all discovered threats for each subsystem

6. Score threats with DREAD:

- Use the above mentioned DREAD scoring for each threat.
- Add up scores to categorize as Critical, High, Medium or Low.

7. Prepare mitigation mapping:

- Propose mitigation strategies for these threats.

• **Applying to the voting subsystems**

1. Online voting client:

- STRIDE will be applied to client login, ballot submission and communication with the server.
- Threats like spoofing, tampering and information disclosure will be prioritized.
- DREAD scoring will highlight which threat requires the strongest mitigation.

2. Vote storage (Database):

- STRIDE will be applied to data at rest and database queries.
- DREAD scoring will show the tampering and DoS are critical.

3. Vote tallying:

- STRIDE will be applied to the aggregation and reporting stage.
- DREAD scoring will emphasize the tampering and repudiation are critical.

Threat analysis and modelling

ID	Threat Description	Subsystem	STRIDE Category	DREAD Score (1-15)	Risk Level
T1	Credential theft via phishing/malware - attacker steals voter login/password and votes on their behalf.	Online Voting Client	Spoofing	38	High
T2	Tampering with client software - malicious modification of the voting client binary or injected JavaScript changes ballot before submission.	Online Voting Client	Tampering	36	High
T3	Replay of captured session token - attacker replays a valid session cookie/token to cast additional votes.	Online Voting Client	Spoofing/Tampering	33	High
T4	Man-in-the-middle (MITM) attack - intercepts vote submission in transit, alters ballot or steals credentials.	Online Voting Client	Tampering/Information Disclosure	40	Critical
T5	Denial-of-service (DoS/DDoS) - attackers flood online portal during election period, preventing voting.	Online Voting Client	Denial of Service	28	High
T6	Vote storage database tampering - attacker modifies or deletes stored votes in backend DB.	Vote Storage & Tallying	Tampering	45	Critical
T7	Insider attack - rogue admin alters tallying algorithm to bias results.	Vote Storage & Tallying	Tampering/Elevation of Privilege	42	Critical
T8	SQL injection in vote submission API - attacker exploits poor input validation to access/alter DB.	Vote Storage & Tallying	Tampering/Elevation of Privilege	40	Critical

T9	Information leakage - sensitive election data exposed via debug logs or misconfigured API.	Vote Storage & Tallying	Information Disclosure	25	High
T10	Repudiation - voter denies casting a vote because logs are insufficient or non-verifiable.	Both	Repudiation	22	Medium
T11	Privilege escalation via misconfigured access control - attacker gains admin rights in tallying service.	Vote Storage & Tallying	Elevation of Privilege	38	High
T12	Ransomware attack on tallying servers - encrypts vote database before results computed.	Vote Storage & Tallying	Denial of Service/Tampering	41	Critical

Prioritizing risks

The combined STRIDE and DREAD method provide a structured approach to risk prioritization.

- STRIDE:
 - Allows to systematically identify threats across all subsystems without overlooking key categories.
- DREAD:
 - Allows to quantitatively rank the threats by severity, considering factors such as potential damage, ease of exploitation and number of affected users.
 - By scoring each threat on a scale from 0 to 50, it enables separation of risks from critical to low.

This ranking enables security resources are focused on the most damaging and exploitable threats first, rather than considering all threats equally.

Improving security of subsystems

- Online voting client:
 - STRIDE revealed spoofing and MITM as major risks and DREAD scoring prioritized strong authentication and end-to-end encryption as necessary defenses.
- Vote storage:

- Database tampering and SQL injection scored as critical, guiding the need for robust input validation, query sanitization and database integrity checks.
- Vote tallying:
 - Insider privilege escalation and algorithm manipulation ranked as critical, highlighting the need for access control, separation of duties and verifiable audit logs.

Integration into SDLC

These steps are used in SDLC to integrate security.

1. Requirement phase:
 - Document security requirements like multi factor authentication, vote encryption and audit logging.
2. Design phase:
 - Conduct DFD and STRIDE workshops, update threat model diagrams.
3. Implementation phase:
 - Use secure coding practices, static analysis (SAST) and dependency scanning.
4. Testing phase:
 - Penetration testing, DAST and red team exercises simulating election day.
5. Deployment phase:
 - Verify configs, monitoring, IDS/IPS and logging of vote transactions.
6. Maintenance phase:
 - Repeat threat modelling after major feature changes and yearly security reviews.

Assumptions: Organization follows an Agile SDLC with integrated security review resembling DevSecOps. Threat modelling sessions are repeated at each sprint for high impact user stories.

Part Two

Threat	Potential Solutions	Pros	Cons	Selected Solution	Justification
T1 - Credential theft via phishing/malware	• Multi-factor authentication (MFA) • Phishing detection training and awareness campaigns • Device fingerprinting & anomaly detection	• MFA greatly reduces credential only risk • Anomaly detection adds resilience	• User inconvenience • Cost of detection systems	MFA with anomaly detection	Provides strong security against stolen passwords while balancing usability.
T2 - Tampering with client software (malicious binary/JS injection)	• Code signing and integrity verification • Regular patching & auto-update mechanism • Content Security Policy (CSP) in browser	• Code signing prevents unauthorized binaries • CSP reduces injection risk	Requires infrastructure to manage certificates and updates	Code signing and CSP	Ensures client software authenticity and prevents injection in web clients.
T3 - Replay of session token	• Use short-lived JWT tokens with refresh • Bind tokens to device/IP • Implement nonce/timestamp in requests	• Nonces stop replay directly • JWT with refresh provides secure lifecycle	Device/IP binding breaks in roaming scenarios	Nonce and short-lived tokens	Directly mitigates replay attacks with minimal usability impact.
T4 - MITM altering vote in transit	• Enforce TLS 1.3 with HSTS • End-to-end vote encryption • Certificate pinning in client app	• TLS/HSTS industry standard • E2E encryption ensures ballot secrecy	• E2E adds complexity • Pinning • causes availability issues on CA rotation	TLS 1.3, HSTS and E2E encryption	TLS covers most MITM risks and E2E ensures no vote can be altered even if TLS fails.

T5 - DoS/DDoS on online voting portal	• Rate limiting & CAPTCHA • Web Application Firewall (WAF) with DDoS mitigation • Geo-distributed load balancing & CDN protection	• WAF/CDN effective against volumetric DDoS • CAPTCHA filters bots	• Costs for CDN • CAPTCHA may hinder accessibility	WAF, CDN and rate limiting	Balanced approach: scalable protection without blocking legitimate users.
T6 - Vote storage DB tampering	• DB encryption with integrity checks • Strict RBAC & least-privilege • Immutable backup with write-once storage	• Integrity chains ensure detection • RBAC reduces insider risk	• Requires strong key mgmt • Performance overhead	DB integrity verification and RBAC	Prevents tampering and ensures detection of unauthorized changes.
T7 - Insider altering tallying algorithm	• Separation of duties • Signed, version-controlled tallying code • Immutable audit logs	• Signed code prevents silent tampering • Dual control reduces insider abuse	More operational overhead	Signed tallying code and audit logs	Ensures any tampering is detectable and attributable.
T8 - SQL Injection in submission API	• Parameterized queries/ORM • Input validation & sanitization • WAF rules for SQLi detection	• Parameterized queries eliminate SQLi • WAF adds fallback	Dev training required	Parameterized queries and validation	Most effective, industry-standard defense against SQLi.
T9 - Information leakage via logs/misc config	• Centralized secure logging with redaction • Regular config review & hardening • Data Loss Prevention (DLP) scanning	• Redaction prevents exposure • Config review reduces risks	DLP adds overhead	Secure logging and config hardening	Prevents both accidental and misconfigured data exposure.

T10 - Repudiation (voter denies casting a vote)	• Cryptographic receipt for each vote • Immutable audit logs • Digital signatures per transaction	• Receipts provide proof • Audit logs prevent manipulation	Receipts may threaten vote secrecy if poorly designed	Receipt-based verifiable voting and audit logs	Balances non-repudiation with secrecy by issuing anonymous confirmation receipts.
T11 - Privilege escalation in tallying service	• Principle of least privilege (PoLP) • Mandatory Access Control • Regular access review & monitoring	MAC policies enforce strict runtime controls	Complex to configure	PoLP and MAC policies	Provides layered defense against privilege abuse.
T12 - Ransomware on tallying servers	• Immutable, offline backups • Application whitelisting & EDR • Database replication with failover cluster	• Backups guarantee recovery • EDR blocks ransomware execution	Backups require regular testing	Immutable backups and EDR	Ensures resilience with both prevention and recovery.

Selected Security Solution	Implementation Strategy
T1 - Credential theft (MFA)	• Implement MFA at login for the Online Voting Client. • Require a second factor (TOTP or SMS OTP) after password entry. • Integrate MFA in both home and polling station clients. • Ensure MFA tokens are validated server-side before granting access.
T2 - Client software tampering (Code Signing & Integrity Check)	• Sign the voting client binaries with a trusted certificate. • Verify signatures on client startup. • Implement runtime integrity checks for JavaScript code to detect injection. • Update verification logic on the web server and polling station terminals.
T3 - Replay attacks (Session)	• Use short-lived session tokens.

Token Management)	• Implement server-side token invalidation after logout or vote submission. • Include unique nonce per transaction to prevent replay. • Apply checks in client submission logic and server API endpoints.
T4 - MITM (TLS & Certificate Pinning)	• Enforce TLS 1.3 for all client-server communication. • Implement certificate pinning on client applications. • Enable HSTS headers on web servers. • Apply encryption and verification on both client app and web server endpoints.
T5 - DoS/DDoS (Rate Limiting & CDN)	• Deploy a Web Application Firewall (WAF) with rate limiting. • Use CDN services to absorb high traffic. • Monitor traffic patterns and trigger alerts for abnormal spikes. • Apply mitigation on server edge and load balancers.
T6 - Database tampering (DB Integrity & Backups)	• Enable database transaction logging and tamper-evident logs. • Regular automated backups with version control. • Apply role-based access controls (RBAC) for DB administrators. • Implement these on database servers and admin access points.
T7 - Insider attack (Separation of Duties & Audit Logging)	• Implement RBAC for all admin functions. • Require multi-party approval for sensitive operations. • Enable audit logging for all admin actions. • Apply controls on tallying server and admin interfaces.
T8 - SQL Injection (Input Validation & Parameterized Queries)	• Use parameterized queries or prepared statements in all vote submission APIs. • Apply strict input validation and sanitization on client and server sides. • Include automated static code analysis for injection vulnerabilities. • Implement this in the backend API layer.
T9 - Information leakage (Secure Logging & Data Masking)	• Mask sensitive data in logs. • Avoid logging PII in debug logs. • Apply access controls on log storage. • Implement logging policies on server side and monitoring tools.
T10 - Repudiation (Digital	• Sign each vote digitally with user credentials. • Maintain immutable audit logs of vote submission. • Ensure logs are verifiable by election officials.

Signatures & Audit Trails)	• Apply in vote storage backend and client-side submission logic.
T11 - Privilege escalation (RBAC & Security Patches)	• Apply least privilege principle to all admin accounts. • Regularly patch OS and applications to fix privilege vulnerabilities. • Monitor access logs for unusual activity. • Apply on tallying servers and admin interfaces.
T12 - Ransomware attack (Backups & Endpoint Protection)	• Implement automated, offline backups of the vote database. • Deploy anti-malware/endpoint protection on tallying servers. • Enable network segmentation to isolate critical systems. • Apply on tallying servers, database servers, and connected admin endpoints.

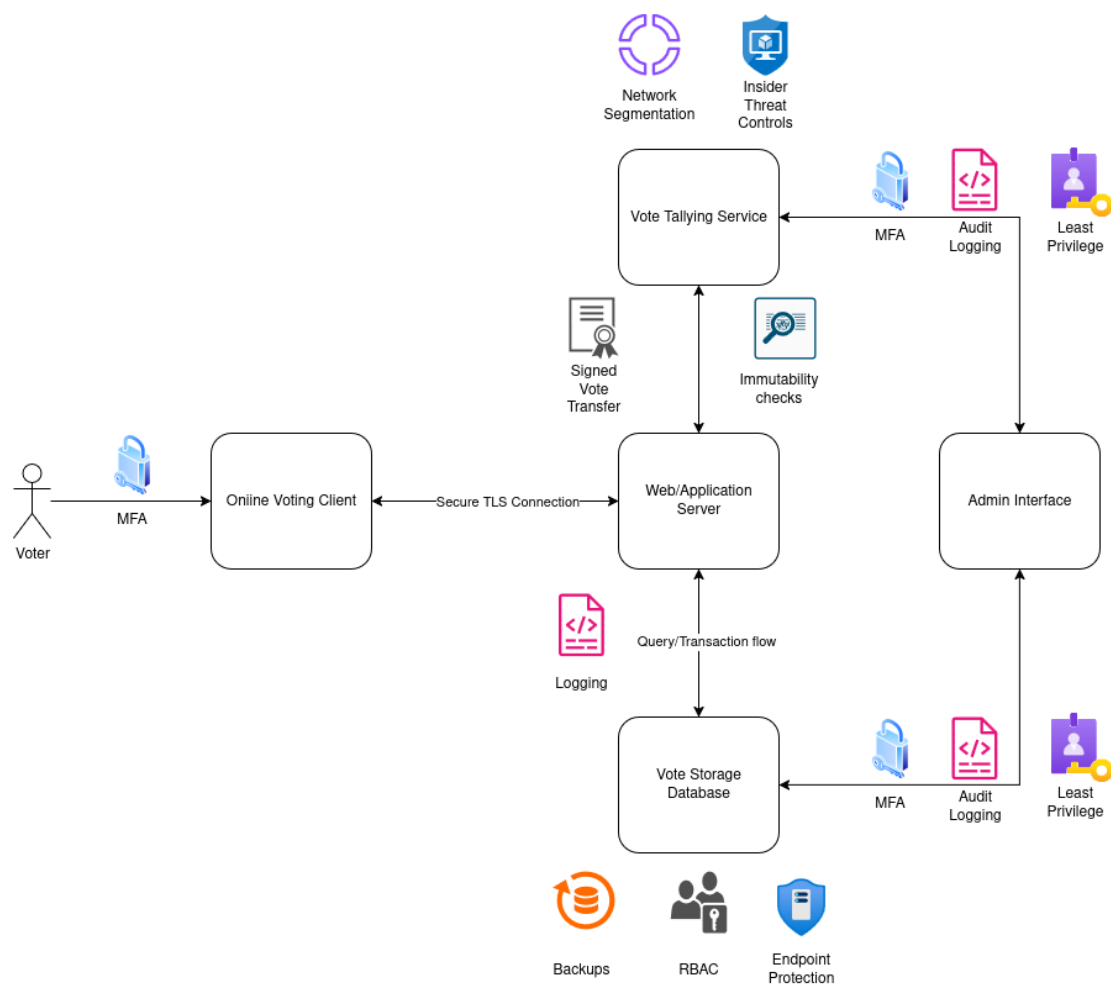


Figure 2.1:

Overall Security Implementation

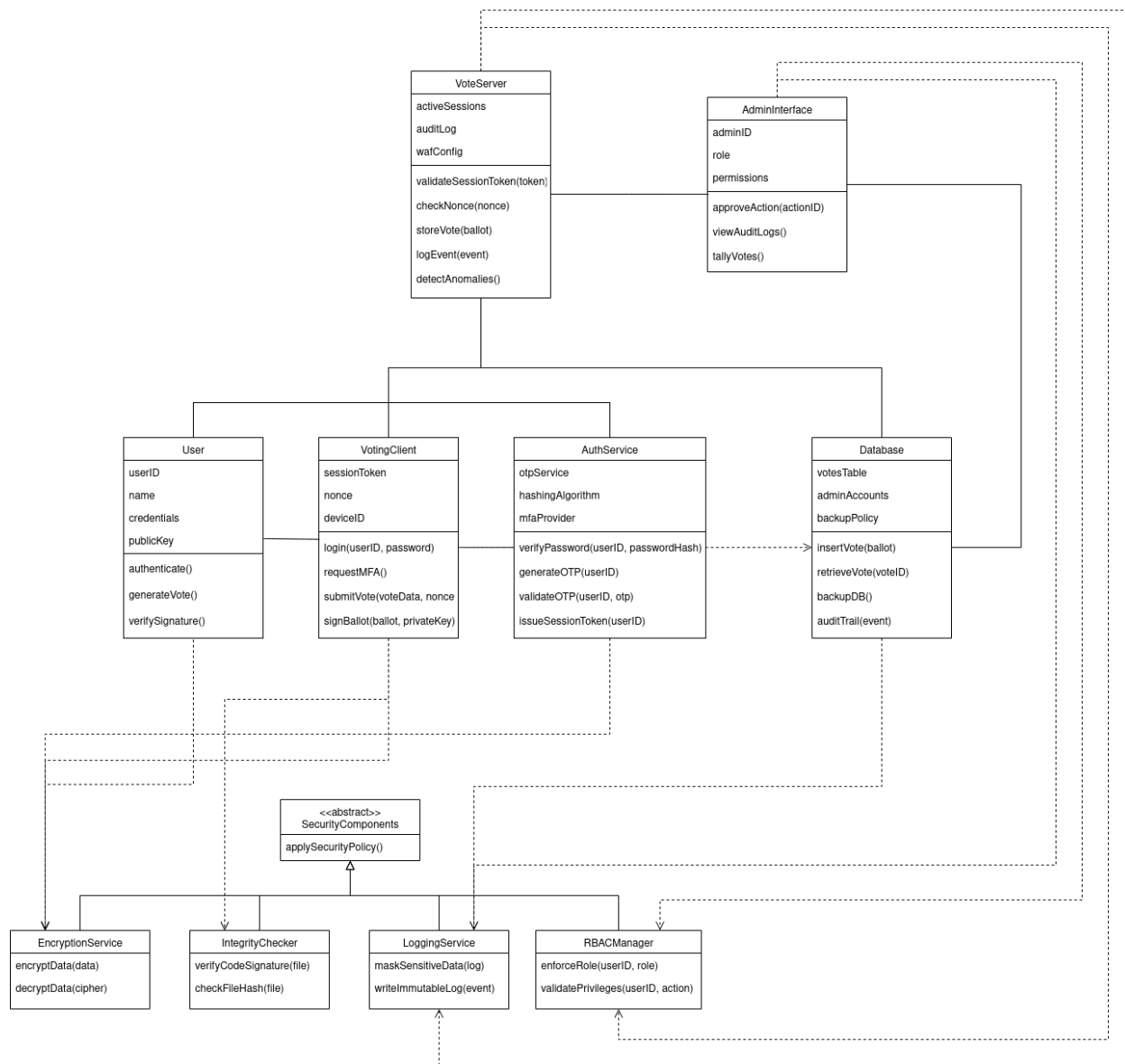


Figure 2.2: Class Diagram

References

- Conklin, L. (2022). *Threat Modeling Process* | OWASP. [online] owasp.org. Available at: https://owasp.org/www-community/Threat_Modeling_Process.
- Drake, V. (2022). *Threat Modeling* | OWASP. [online] owasp.org. Available at: https://owasp.org/www-community/Threat_Modeling.
- Misbah Thevarmannil (2023). *10 Types of Threat Modeling Methodology To Use in 2025 - Practical DevSecOps*. [online] Practical DevSecOps. Available at: https://www.practical-devsecops.com/types-of-threat-modeling-methodology/?srsltid=AfmBOorlPKTijRA_TsenVvvxNvbnr4VW9mrVTrdssv9vrQydcqZ40DHX [Accessed 12 Sep. 2025].
- Kirtley, N. (2023). *DREAD Threat Modeling* - Threat-Modeling.com. [online] Threat-Modeling.com. Available at: <https://threat-modeling.com/dread-threat-modeling/>.
- Content Studio (2024). *What Is the DREAD Threat Model?* [online] Purestorage.com. Available at: <https://www.purestorage.com/knowledge/dread-threat-model.html>.
- EC-Council (2022). *DREAD Threat Modeling: An Introduction to Qualitative Risk Analysis*. [online] Cybersecurity Exchange. Available at: <https://www.eccouncil.org/cybersecurity-exchange/threat-intelligence/dread-threat-modeling-intro/>.
- Misbah Thevarmannil (2022). *What Is the STRIDE Threat Model? Beginner's Guide - 2025 - Practical DevSecOps*. [online] Practical DevSecOps. Available at: <https://www.practical-devsecops.com/what-is-stride-threat-model/?srsltid=AfmBOoWdtnK4KS43rrD640sGvOLzFGSnpAJz-M0Js1Xo8KOnGSvMjXZ> [Accessed 12 Sep. 2025].
- GeeksforGeeks (2025). *Software Development Life Cycle (SDLC)*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/software-engineering/software-development-life-cycle-sdlc/>.
- Creately.com. (2025). *DFD E Voting system lvl 1 [classic]* | Creately. [online] Available at: <https://creately.com/diagram/example/iypoz7qu/dfd-e-voting-system-lvl-1-classic> [Accessed 13 Sep. 2025].
- Quist, S.C., AMEGATSE, L.K. and Dickson, D. (2016). Conceptual Design of E-Voting System for Academic Institution. *International Journal for Innovation Education and Research*, [online] 4(12), pp.96–109. doi:<https://doi.org/10.31686/ijer.vol4.iss12.54>.
- Visual Paradigm (2024). *UML Class Diagram Tutorial*. [online] Visual-paradigm.com. Available at: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>.
- www.softwareideas.net. (n.d.). *UML Abstract Class - Using Abstract Modifier* - Software Ideas Modeler. [online] Available at: <https://www.softwareideas.net/uml-abstract-class>.
- Mueller, N. (n.d.). *Credential stuffing Software Attack* | OWASP Foundation. [online] owasp.org. Available at: https://owasp.org/www-community/attacks/Credential_stuffing.
- Carson, J. (2024). *Credential theft: How your organization's credentials get compromised and what to do about it*. [online] Delinea. Available at: <https://delinea.com/blog/credential-theft>.
- OWASP (n.d.). *Web Parameter Tampering* | OWASP. [online] owasp.org. Available at: https://owasp.org/www-community/attacks/Web_Parameter_Tampering.
- Learning Center. (n.d.). *Parameter Tampering* | Imperva. [online] Available at: <https://www.imperva.com/learn/application-security/parameter-tampering/>.
- GeeksforGeeks (2022). *What are Session Replay Attacks?* [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/ethical-hacking/what-are-session-replay-attacks/>.

- OWASP (2020). *Session hijacking attack*. [online] Owasp.org. Available at: https://owasp.org/www-community/attacks/Session_hijacking_attack.
- Imperva (2025). *What is MITM (Man in the Middle) Attack*. [online] Imperva. Available at: <https://www.imperva.com/learn/application-security/man-in-the-middle-attack-mitm/>.
- Malik, F. (2024). *How to Prevent Man-in-the-Middle Attacks: 10 Techniques | StrongDM*. [online] discover.strongdm.com. Available at: <https://www.strongdm.com/blog/man-in-the-middle-attack-prevention>.
- Imperva (2021). *DDoS Attack Types & Mitigation Methods | Imperva*. [online] Learning Center. Available at: <https://www.imperva.com/learn/ddos/ddos-attacks/>.
- Cloudflare (2019). *Cloudflare*. [online] Cloudflare. Available at: <https://www.cloudflare.com/learning/ddos/ddos-mitigation/>.
- Imperva (n.d.). *What is SQL Injection | SQLI Attack Example & Prevention Methods | Imperva*. [online] Imperva. Available at: <https://www.imperva.com/learn/application-security/sql-injection-sqli/>.
- OWASP (2024). *SQL Injection*. [online] OWASP. Available at: https://owasp.org/www-community/attacks/SQL_Injection.
- Cloudflare (2024). *How to prevent SQL injection*. [online] Cloudflare.com. Available at: <https://www.cloudflare.com/learning/security/threats/how-to-prevent-sql-injection/>.
- OWASP (n.d.). *Repudiation Attack Software Attack | OWASP Foundation*. [online] owasp.org. Available at: https://owasp.org/www-community/attacks/Repudiation_Attack.
- Bangalore, K. (2023). *Privilege Escalation Attack & its Mitigation*. [online] Medium. Available at: <https://medium.com/@kavib/privilege-escalation-its-mitigation-7359e9bad21d>.
- Team, S. (2025). *Linux Privilege Escalation: Techniques, Prevention & More*. [online] Strongdm.com. Available at: <https://www.strongdm.com/blog/linux-privilege-escalation>.
- NCSC (2020). *Mitigating malware and ransomware attacks*. [online] www.ncsc.gov.uk. Available at: <https://www.ncsc.gov.uk/guidance/mitigating-malware-and-ransomware-attacks>.
- CISA (2025). *Ransomware Guide | CISA*. [online] www.cisa.gov. Available at: <https://www.cisa.gov/stopransomware/ransomware-guide>.